

RE Leçon 2 — GDB : Déboguer et inspecter un binaire

Qu'est-ce que GDB ?

GDB (GNU Debugger) permet d'exécuter un programme **pas à pas** et d'inspecter la mémoire, les registres et la pile en temps réel.

En reverse engineering, GDB est utilisé pour l'**analyse dynamique** : on observe le programme pendant qu'il tourne pour comprendre ce qu'il fait.

Analyse statique (Ghidra)

|
lire le code
sans lancer
le programme

Analyse dynamique (GDB)

|
exécuter pas à pas
voir les vraies valeurs
en mémoire au runtime

Setup — compiler avec symboles de debug

Sans symboles, GDB affiche de l'assembleur brut. Avec **-g**, GDB peut afficher le code C ligne par ligne.

```
# Sans symboles (ce qu'on analyse en RE réel)
gcc -o binary source.c

# Avec symboles (pour apprendre)
gcc -g -O0 -o binary source.c
```

-O0 désactive les optimisations du compilateur — le code généré est plus proche de ce qu'on a écrit, plus facile à lire.

1. Lancer GDB

```
gdb ./binary
```

Pour passer des arguments au programme :

```
gdb --args ./binary argument1 argument2
```

2. Commandes fondamentales

Commande	Raccourci	Rôle
<code>run</code>	<code>r</code>	Lancer le programme
<code>quit</code>	<code>q</code>	Quitter GDB
<code>break main</code>	<code>b main</code>	Poser un breakpoint sur <code>main</code>
<code>break 12</code>	<code>b 12</code>	Breakpoint à la ligne 12
<code>next</code>	<code>n</code>	Exécuter la ligne suivante (sans entrer dans les fonctions)
<code>step</code>	<code>s</code>	Exécuter la ligne suivante (entre dans les fonctions)
<code>continue</code>	<code>c</code>	Continuer jusqu'au prochain breakpoint
<code>print x</code>	<code>p x</code>	Afficher la valeur de la variable <code>x</code>
<code>info registers</code>	<code>i r</code>	Afficher tous les registres
<code>backtrace</code>	<code>bt</code>	Afficher la pile d'appels
<code>list</code>	<code>l</code>	Afficher le code source autour de la position courante
<code>disassemble</code>	<code>disas</code>	Désassembler la fonction courante

3. Breakpoints

Un breakpoint = point d'arrêt. Le programme s'arrête là et te rend la main.

```
(gdb) break main           # arrêt au début de main
(gdb) break 15             # arrêt à la ligne 15
(gdb) break check_password # arrêt au début de la fonction
(gdb) info breakpoints     # lister tous les breakpoints
(gdb) delete 1             # supprimer le breakpoint n°1
```

4. Inspecter les variables

```
(gdb) print x              # valeur de x
(gdb) print &x            # adresse de x
(gdb) print *p            # valeur pointée par p
(gdb) print/x x           # valeur de x en hexadécimal
(gdb) print/s buffer      # afficher comme string
```

5. Inspecter la mémoire

```
(gdb) x/10x $rsp          # 10 valeurs hex à partir du stack pointer
(gdb) x/s 0x402000        # lire une string à l'adresse 0x402000
```

```
(gdb) x/i $rip          # instruction courante
(gdb) x/20i main        # 20 instructions depuis main
```

Format : `x/<nombre><format> <adresse>`

Format	Signification
<code>x</code>	hexadécimal
<code>d</code>	décimal
<code>s</code>	string
<code>i</code>	instruction assembleur
<code>b</code>	byte (1 octet)
<code>w</code>	word (4 octets)
<code>g</code>	giant (8 octets)

6. Inspecter les registres

```
(gdb) info registers      # tous les registres
(gdb) print $rax          # valeur de rax
(gdb) print $rsp          # stack pointer
(gdb) print $rip          # instruction pointer (position courante)
```

Registres x64 importants :

Registre	Rôle
<code>rip</code>	Adresse de l'instruction courante
<code>rsp</code>	Sommet de la pile (stack pointer)
<code>rbp</code>	Base de la frame courante
<code>rax</code>	Valeur de retour des fonctions
<code>rdi</code>	1er argument d'une fonction
<code>rsi</code>	2ème argument
<code>rdx</code>	3ème argument

7. Modifier des valeurs à la volée

```
(gdb) set variable x = 999      # changer la valeur de x
(gdb) set $rax = 1              # changer un registre
```

Utile en RE pour tester des chemins d'exécution sans modifier le code.

8. Désassembler

```
(gdb) disassemble main          # désassembler main
(gdb) disassemble check_password # désassembler une fonction
```

Pour afficher les adresses et le code source en même temps :

```
(gdb) layout asm      # vue assembleur
(gdb) layout src      # vue code source
(gdb) layout regs     # vue registres
```

Exemple complet — analyser crackme1

On reprend le `crackme1` de l'exercice 8 (leçon 1).

Recompile avec symboles de debug :

```
gcc -g -O0 -o crackme1_debug crackme1.c
gdb ./crackme1_debug
```

Dans GDB :

```
# Poser un breakpoint sur main
(gdb) break main
(gdb) run

# Le programme s'arrête au début de main
# Avancer ligne par ligne
(gdb) next
(gdb) next

# Quand on arrive sur strcmp, voir ce qui est comparé
(gdb) break strcmp
(gdb) continue

# Inspecter les arguments de strcmp
(gdb) print $rdi      # 1er argument : notre input
```

```
(gdb) print $rsi    # 2ème argument : le mot de passe attendu
(gdb) x/s $rsi      # afficher comme string → on voit "m4tr1x"
```

Sans regarder le code source, on vient de trouver le mot de passe en interceptant l'appel à `strcmp` dans GDB.

Leçons apprises

- GDB = microscope pour observer un programme en cours d'exécution
- `break strcmp + x/s $rsi` = technique classique pour trouver un mot de passe
- `-g -O0` lors de la compilation facilite l'analyse dans GDB
- `layout asm + layout regs` = vue complète pour l'analyse

Prochaine leçon

Leçon 3 — Assembleur x86/x64 : lire l'assembleur généré par GCC, comprendre les patterns `if`, `boucle`, `appel de fonction` en asm.